

ARTI404 – Image Processing

Lab 4: Intensity Transformations and Filtering – Spatial Domain

Student Name	Nawaf Khaled Alwabel
Student ID	2240001111

Imports

```
import matplotlib.pyplot as plt
import numpy as np
from skimage import data, exposure
from skimage.exposure import match_histograms
```

Task 1

Using the same 'moon image', rescale intensity values to include all the intensities that fall within the 3rd and 80th percentiles, and plot the histogram.

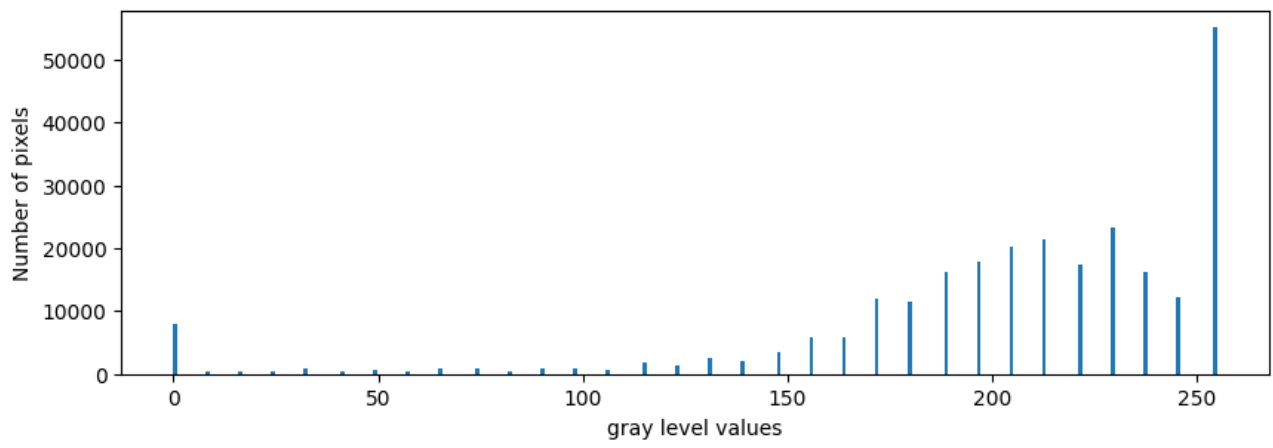
Solution:

```
img = data.moon()
p3, p80 = np.percentile(img, (3, 80))
img_rescale = exposure.rescale_intensity(img, in_range=(p3, p80))

fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap='gray')
plt.axis('off')
plt.title('Contrast stretch (3rd - 80th percentiles)')
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins=256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Output:

Contrast stretch (3rd - 80th percentiles)



Task 2

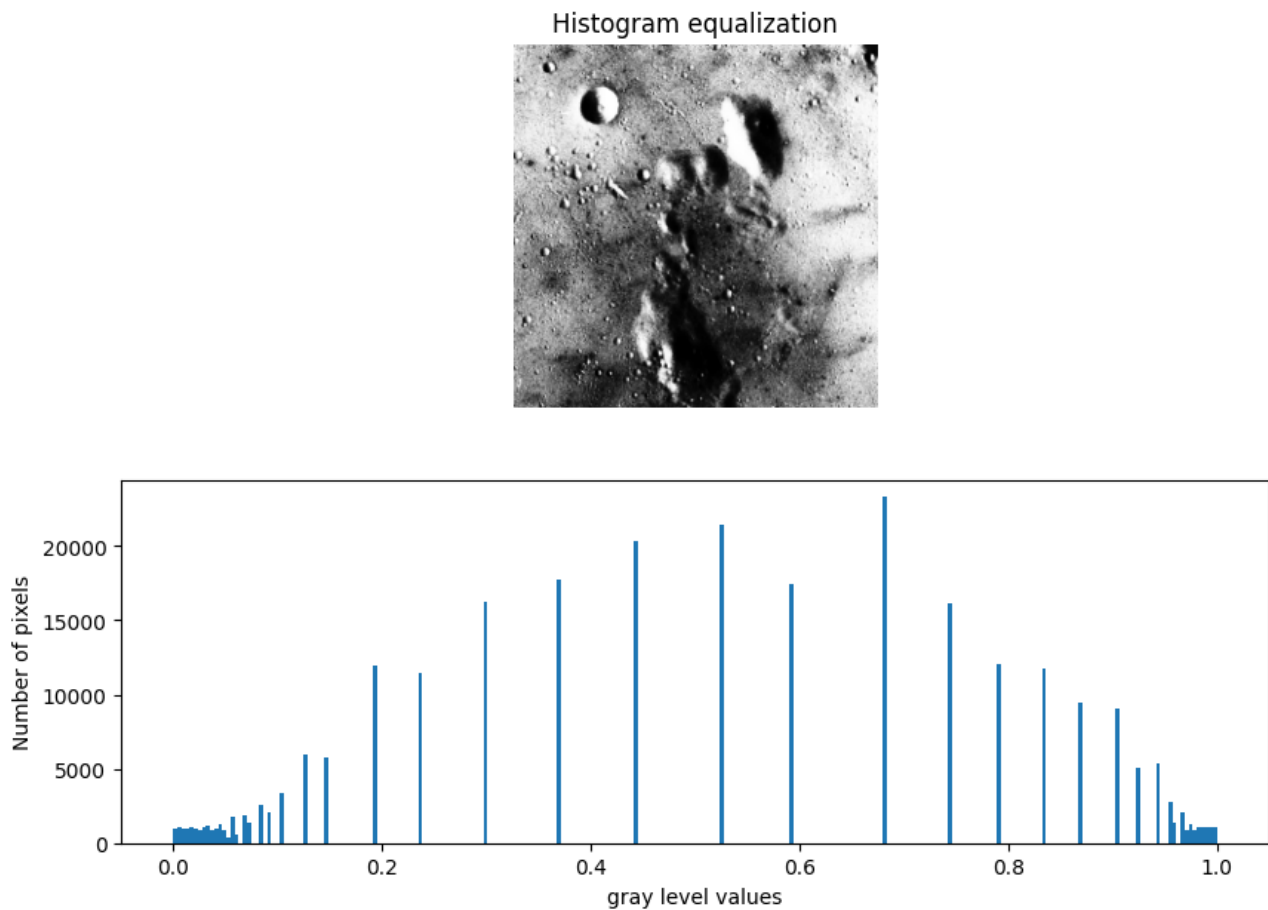
Using the same 'moon image' and the `exposure.equalize_hist` function, display the image and the histogram of the image after flattening the histogram.

Solution:

```
img_eq = exposure.equalize_hist(img)

fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_eq, cmap='gray')
plt.axis('off')
plt.title('Histogram equalization')
fig.add_subplot(2, 1, 2)
plt.hist(img_eq.flat, bins=256, range=(0, 1))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Output:



Task 3

Use the rocket (as reference) and Chelsea images (from skimage.data) and implement histogram matching.

Solution:

```
reference = data.rocket()
image = data.chelsea()
matched = match_histograms(image, reference, channel_axis=-1)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
titles = ['Source (Chelsea)', 'Reference (Rocket)', 'Matched']
for ax, im, title in zip(axes, [image, reference, matched], titles):
    ax.imshow(im)
    ax.set_title(title)
    ax.axis('off')
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(3, 3, figsize=(12, 9))
for i, im in enumerate([image, reference, matched]):
    for c, color in enumerate(['red', 'green', 'blue']):
        hist, bins = exposure.histogram(im[..., c], source_range='dtype')
        axes[c, i].plot(bins, hist / hist.max(), color=color)
        img_cdf, bins = exposure.cumulative_distribution(im[..., c])
        axes[c, i].plot(bins, img_cdf, 'k')
    axes[0, i].set_title(['Source', 'Reference', 'Matched'][i])
plt.tight_layout()
plt.show()
```

Output:



Output:

